

How Agile Practices Impact Customer Responsiveness and Development Success: A Field Study

Jan Recker, *QUT Business School, Queensland University of Technology, Brisbane, Australia*

Roland Holten, *Faculty of Economics and Business Administration, Goethe University Frankfurt, Germany*

Markus Hummel, *Senacor Technologies AG, Eschborn, Germany*

Christoph Rosenkranz, *Faculty of Economics, Business and Social Sciences, University of Cologne, Germany*

ABSTRACT ■

Agile information systems development methods have become popular; however, which specific agile practice to use remains unclear. We argue that three types of agile practices exist—for management, development, and standards—which affect the customer responsiveness of software teams differently. We examine this theory in a field study of a large organization. We find that agile practices improve software team response effectiveness or efficiency, but not both. Agile standards do not improve response mechanisms but are still important to successful information systems development. Our findings help discriminating agile practices and yield insights into how information development projects should be managed.

KEYWORDS: information systems development; agility; agile practices; customer responsiveness; field study; panel survey

INTRODUCTION ■

Agile methods for information systems development, such as Extreme Programming (Beck, 1999) or Scrum (Schwaber & Beedle, 2002), have become popular in today's software industry because they reportedly provide ways to improve an information systems development team's flexibility in terms of the ability to embrace and respond efficiently and effectively to changing requirements (Conboy, 2009; Lee & Xia, 2010; MacCormack, Verganti, & Iansiti, 2001).

In the literature, two streams of research inform the current understanding of agile information systems development. One stream has focused largely on the management of *agile teams* (Lee & Xia, 2010; McHugh, Conboy, & Lang, 2014; Whitworth & Biddle, 2007). This research suggests that agile information systems development can lead to successful development if and when teams are autonomous and diverse (Lee & Xia, 2010) or cohesive and motivated (Whitworth & Biddle, 2007). A second stream of research has examined *agile methodologies* and their use (Erickson, Lyytinen, & Siau, 2005; Maruping, Venkatesh, & Agarwal, 2009; Maruping, Zhang, & Venkatesh, 2009; Pikkarainen, Haikara, Salo, Abrahamsson, & Still, 2008). This work shows, for instance, that different cultures favor different agile methods (Iivari & Iivari, 2011), and that agile methods have a positive impact on project success (Serrador & Pinto, 2015).

Our ambition is to integrate the two streams of research. We believe this move is *important* because it allows for understanding of the interdependencies between the social (teams—the structure) and the technical (agile tools and methodological procedures—the technology and tasks) in the sociotechnical work that is information systems development information systems development (Bostrom, Gupta, & Thomas, 2009; Winter, Berente, Howison, & Butler, 2014). We also believe this move is *timely*, because an understanding of the interdependencies between the social and technical components of information systems development requires a thorough understanding of each component, which is provided in isolation by the two existing streams in the literature. In making this move we can examine the contributions of each component relative to each other (i.e., the social versus the technical) and also evaluate the contribution offered by the joining of the components (i.e., the socio-technical).

How Agile Practices Impact Customer Responsiveness and Development Success

Specifically, we focus on how different agile practices (the technical component) affect customer responsiveness of agile teams (the social component). This has not yet been addressed in the existing literature. By making this move, we can also address an ecologically fundamental problem of agile development, that of method comparison (Conboy, 2009): What are the effects of different agile practices? And, which agile practices from which available methodologies should be employed in information systems development projects? While statements such as those in the *Agile Manifesto* (Fowler & Highsmith, 2001) portray agile information systems development as a coherent, simple, and clear concept, the reality of agile is much different, much messier (Conboy, 2009), and requires interpretation of what is essential and what is less essential (Iivari & Iivari, 2011).

We will show that the problem of selecting agile practices is far from simple, yet of significant value to the literature and practice: For the literature on agility, differentiating between varying types and components is helpful, as it helps to identify subsets of practices (Ågerfalk, Fitzgerald, & Slaughter, 2009) with different effects on different component parts of agility (Conboy, 2009). For the practice on agility, this move yields ecological and pragmatic value to organizations. Companies typically have more design freedom in selecting a set of practices—the technical component of agile—which they can employ, train, or in case discard, than they have in selecting the structure and composition of information systems development teams—the social component of agile.

To meet our objective, we start with the model of information systems development agility proposed by Lee and Xia (2010), which focuses on the effectiveness and efficiency of agile teams' responses to changes demanded by customers. We then extend this model by identifying different types of agile practices, viz., development practices, management practices, and agile standards and norms. We then examine how this

extended model informs the agility of information systems development teams and information systems development success. We examine our research model using panel data gathered in a field study from a sizable agile development initiative within a large retail organization.

Our research contributes to the literature in several ways: Theoretically, we provide an extended model of information systems development agility that draws attention to different types of agile practices and their relative influence on software team response effectiveness and efficiency. This is an important extension to the literature, which thus far has examined team characteristics as antecedents to agility (Lee & Xia, 2010). Empirically, we compare different agile practices with different agile methodologies in terms of how they contribute to information systems development success, in turn providing empirical evidence about the efficacy of different agile practices, which helps to discriminate the "methodology jungle" (Conboy, 2009).

We proceed as follows. First we review the prior research on agile information systems development and the different constituent practices that characterize agility. Next we describe our research model and develop our propositions. We then provide a description of the research methods used and examine our data. We discuss findings and implications and then provide a review of limitations and contributions.

Prior Research

Agile Information Systems Development

Agile information systems development approaches such as Scrum (Schwaber & Beedle, 2002), Extreme Programming (Beck, 1999), and Crystal (Cockburn, 2001) purportedly provide ways to improve a team's agility in terms of the ability to embrace and respond to changing requirements (Conboy, 2009; Lee & Xia, 2010). Following Lee and Xia (2010), we define *information systems development agility* as a development team's ability to efficiently and effectively respond to user or customer requirement changes

during an information systems development project life cycle. *Response extensiveness* describes the extent, range, scope, or variety of software team responses to customer inquiries, and *response efficiency* relates to the time, cost, resources, or effort associated with software team responses (Lee & Xia, 2010, p. 90).

The key elements of agile approaches to information systems development usually include close collaboration among stakeholders, intensive iteration and rapid prototyping, openness to changing requirements, and the eschewal of heavyweight processes and formal documentation (Conboy, 2009). Thus, the label of agile information systems development is an umbrella term for a number of distinct methodologies organized around some common principles. In a literature review (Hummel, Rosenkranz, & Holten, 2013), we found that some of these principles and practices (such as pair programming or onsite customers) have received significant attention in the literature, whereas other practices (such as collective code ownership or continuous integration) have not. Moreover, the impact on agility and the relative value of those practices that have been examined in the literature (e.g., pair programming) remains inconclusive (Balijepally, Mahapatra, Nerur, & Price, 2009), signifying a lack of clarity in our theoretical understanding of agility. This has been also alluded to as the missing "theoretical glue" (Conboy, 2009, p. 330) of agile information systems development. Yet, independent of the inconclusive and imbalanced academic debate over the value of different agile practices, the industry practice of information systems development has seen the widespread adoption of the different available methodologies in recent years (Dingsøyr, Nerur, Balijepally, & Moe, 2012; VersionOne, 2015).

Two lines of argument inform the current understanding of a nascent 'theory of agile.' The first, proposed by Lee and Xia (2010), argues that agile information systems development is fundamentally people-centric and recognizes the value of team members' competencies in

bringing agility to information systems development processes; therefore, the team's characteristics are important (Lee & Xia, 2010, p. 90). This research has shown that two factors affect response extensiveness and response efficiency: (1) *team diversity*—the heterogeneity within the team in terms of individual attributes, such as age, gender, ethnic background, education, functional background, tenure, and technical abilities; and (2) *team autonomy*—the degree of discretion and independence granted to the team in scheduling the work, determining the procedures and methods to be used, selecting and deploying resources, hiring and firing team members, assigning tasks to team members, and carrying out assigned tasks.

A second line of research in the literature, which has proceeded quite independently, focuses on the behavior and management of information systems development teams in terms of how they can be both enabled and restricted in their autonomy and flexibility to respond to changing requirements (Maruping, Venkatesh, & Agarwal, 2009), and how they can be enabled to coordinate (Maruping, Zhang, & Venkatesh, 2009).

Both lines of research have in common that the primary unit of interest is the *team* that is using agile methodologies. While we agree that team characteristics are important, these usually are seen as input factors to the information systems development process (Siau, Long, & Ling, 2010), and as such, rather static and less amenable to change for an information systems development project at any given point in time. For example, it is hard to conceive that information systems development managers can choose anything or do much about team diversity with given and fixed resources in terms of available team members within a company. The same holds for team autonomy—one cannot simply command it, so how does one bring about independence and self-organization within a team, without a considerable allowance of time?

The second problem in the current level of understanding pertains to a lack

of integration between the two streams: both lines of research examine agile information systems development from a team angle but choose different lenses and measures to do so. For example, the study by Lee and Xia (2010) examined information systems development success in terms of software functionality and completion on-time and on-budget, whereas Maruping, Venkatesh, and Agarwal (2009) and Maruping, Zhang, and Venkatesh (2009) used project quality in terms of bug severity and software complexity as measures.

We argue that it is both feasible and prudent to integrate these two primary views on agility, and to extend their primary unit of interest toward more salient choice factors: the question of which methodology or practice to use in agile information systems development (Conboy, 2009).

The Differences Between Agile Practices

Several agile information systems development approaches are described in textbooks (e.g., Beck, 1999; Schwaber & Beedle, 2002), covering rules to follow in terms of the actions and outcomes that are required, prohibited, or permitted.

Agile information systems development approaches-in-use, however, are often not followed by the book; rather they can be characterized as a combination of practices taken from different methodologies (Abrahamsson, Salo, Ronkainen, & Warsta, 2002; Fitzgerald, Hartnett, & Conboy, 2006). The diversity of agile methodologies, along with patterns of common components and practices across them, therefore suggests that information systems development is not a question of approach (Wang, Conboy, & Cawley, 2012), method (Conboy, 2009), or methodology (Maruping, Venkatesh, & Agarwal, 2009), but rather a question of *practices* that team members can choose to enact as a methodology-in-use (Fitzgerald, 1997).

Different practices from different approaches are often chosen at the start of a project to tailor the methodology to specific project settings (Fitzgerald et al.,

2006). Inevitably, this has led to the emergence, adoption, and use of multiple variations or implementations of 'textbook' methodologies. In turn, different agile practices are employed in any given project, which are parts of one or more 'textbook' methodologies (Berente, Hansen, & Rosenkranz, 2015). In essence, that makes the selection of which agile practices to use a choice problem for practitioners.

Much in the same way that agile-in-use differs between organizations (VersionOne, 2015), the scholarly attention devoted to the different available practices also varies and some practices are viewed as more prominent. For example, literature such as those provided by Hummel et al. (2013) and Dingsøyr et al. (2012) show that there is a large body of work on pair programming (Balijepally et al., 2009) but relatively little work on collective code ownership (Maruping, Zhang, & Venkatesh, 2009). What remains missing is an understanding of the various agile practices in terms of (1) their specific effects on agility, (2) their relative value, and (3) their comparative benefits for information systems development (Ågerfalk et al., 2009). This situation leads to the challenge of parsimony (Conboy, 2009) in that many practices likely share redundancies and duplication. Still, practices remain the key design element for organizations and management and are the most salient factor in designing interventions and guidance for how to engage in successful information systems development. In the following section, we propose a classification for the differentiation of agile practices based on the mode and locus of control these practices provide.

Theory Development

To overcome the challenge of parsimony (Conboy, 2009), the means that allow for discrimination of agile practices need to be discovered. We suggest that one purposeful way of discrimination is examining the ways in which agile practices provide the means to maintain *control* over information systems

How Agile Practices Impact Customer Responsiveness and Development Success

development. Our basic assumption is that, even though agile information systems development promotes flexibility and autonomy, the practices by which agility is enacted still provide some degree of structure or constraints to project teams, which is important in the management of team work (Barki & Hartwick, 2001) to avoid detrimental outcomes due to individual volition (Cohen & Bailey, 1997) or “free-wheeling” (Boehm, 2002). In other words, agile information systems development does not equate with methodological or managerial anarchy.

Instead, much like the traditional approaches to software development or project management, agile practices, while promoting certain team work properties such as flexibility, autonomy, and self-regulation, still encompass means that ensure that “individuals working on projects act according to an agreed-upon strategy to achieve desired objectives” (Kirsch, 1996, p. 1). Pair programming, for example, provides fairly strict and precise guidelines about how individuals should collaborate in code development (Beck, 1999, p. 58): *“There are two roles in each pair. One partner, the one with the keyboard and the mouse, is thinking about the best way to implement this method right here. The other partner is thinking more strategically: Is this whole approach going to work?”* Other practices such as stand-up meetings, sprint planning, or backlog grooming also follow clear behavioral and procedural norms and rules for how these engagements should proceed.

Through examining how guidelines and rules are embodied in agile practices, and falling back on other approaches that try to categorize practices (Bourque & Fairley, 2014; Jacobson, Ng, McMahon, Spence, & Lidman, 2012), we suggest that at least three sets of agile practices be distinguished. Table 1 summarizes our definitions of the types of agile practices, which we discuss in turn:

1. One set of agile practices primarily sets out to support the **management** of information systems development projects. Information systems development is a significant endeavor that typically takes many weeks to complete, affects many different people (the stakeholders), and involves a development team (rather than a single developer). Any practical method must describe a set of practices to effectively plan, lead, and monitor the efforts of the team. Practices in this set seek to describe and enforce behaviors, processes, and procedures that must be followed during information systems development projects, and that concerns how the projects as such unfold. Typical examples are the practice of daily *stand-up meetings* (STM) from Scrum, enforcing high communication frequency between team members in short meetings (Pikkarainen et al., 2008), and the practice of *small releases* (SR) from Extreme Programming, which enables rapid feedback from customers (Xiaohu, Xu, He, & Maddineni, 2004).

2. A second set of practices focuses on the **development** element and ensures developers’ autonomy, specifying rules of self-regulation and self-monitoring. In the context of information systems development, “development work” is everything that the team does to meet the goals of producing an IS matching the requirements and addressing the opportunity presented by the stakeholders. These practices provide autonomy to single developers and rules about how to monitor and regulate their actions. Although these practices require intense interaction with one or more peer developers, their focus is nonetheless on constraining the actions of the individual developer, and therefore on the self-regulation of goals and self-monitoring of progress. So the focus of development practices is on self-control and the tasks for software development rather than procedures that affect the progression of a project itself. For example, practices from Extreme Programming in this group are *pair programming* (PP), prescribing the interaction of coder and reviewer in a team of peers at one workstation (Vidgen & Wang, 2009), and *continuous code integration* (CCI), enforcing regular and rigorous feedback on new code integrated into the whole system’s code (Xiaohu et al., 2004).
3. A third set of practices prescribes **standards and norms** that the whole team must follow. The development

Type of Agile Practice	Definition	Examples of Agile Practices
Management practices	Rules and procedures for joint discussions by providing behaviors, processes, and artifacts that must be followed in team meetings	Daily stand-up meetings; small releases
Development practices	Rules that provide autonomy to developers to determine themselves what actions are required and how to execute them and which emphasize self-regulation of goals, processes, and progress	Pair programming; continuous code integration
Standards and norms	Stipulation of acceptable team behaviors by sharing development standards and norms of artifacts and components (code) and reinforcing shared behavior through shared objects	Collective code ownership; coding standards

Table 1: Types of agile practices with examples.

work is guided by these practices, which make up the team's ways of working. The team evolves their way of working alongside their understanding of their mission and working environment. As their work proceeds, they continually reflect on their way of working and adapt it to their current context, if necessary. These practices socialize certain norms among team members (e.g., responsibility for code written by others and giving peers the responsibility for their own code) and reinforce shared rituals and experiences (e.g., thinking about side effects their own code might have). For example, *coding standards* (CS), which prescribe rules all developers have to follow when developing code (Hazzan & Dubinsky, 2003), and *collective code ownership* (CCO), which gives each team member responsibility for all the codes (Xiaohu et al., 2004), belong to this set of practices.

Propositions

We now develop a research model to evaluate our assertion that agile

practices can be evaluated in terms of how they affect agility understood as response extensiveness and efficiency, and how they influence the outcomes of agile development. Figure 1 illustrates this model.

The research model has three components. First, we follow Lee and Xia (2010) and define **information systems development success** as the relevant dependent variable of agile development. We conceptualize success in three dimensions. First, *software functionality*, as the extent to which the delivered software meets functional goals and end-user requirements (Lee & Xia, 2010; Weitzel & Graen, 1989). Second, *process performance*, to provide a resource-oriented view on the on-time and on-budget completion of the project (Wallace & Keil, 2004). Third, *customer satisfaction*, because it is one of the fundamental principles of agile information systems development as advocated in the *Agile Manifesto* (Fowler & Highsmith, 2001).

Second, we again follow Lee and Xia (2010) and define **agility** as the ability of agile teams to extensively and

efficiently respond to changing customer requirements.

Third, we distinguish **management practices** from **development practices** and **agile standards and norms** as discussed in Table 1.

We now discuss propositions that link the concepts in our research model. We start by positing that agility positively impacts information systems development success. Successful information systems development largely depends on meeting customer requirements in the delivery of software. If responses made to changes of customer requirements are extensive, it implies that they include broad and detailed information about how requirements translate to software functionality. This in turn should also lead to elevated customer satisfaction. Responding efficiently means that development teams can swiftly alter software when requirements change, in turn increasing performance of the development process. Together, the more agile a team operates, the more successful the outcomes of the projects should be and we therefore propose:

P1: Agility positively affects information systems development success.

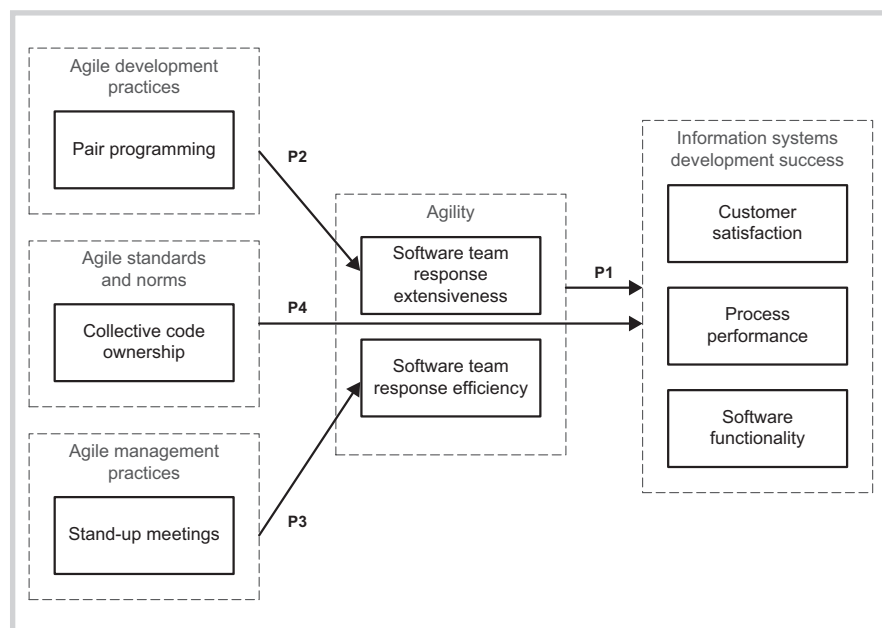


Figure 1: Research model.

Next, we examine how agility is influenced by the three different kinds of agile practices we propose. First, agile **development practices**, such as pair programming and continuous code integration, provide guidelines for individuals to focus on software testing, simplifying code, or enhancing code quality through peer review. Importantly, through these mechanisms of self-control and autonomy, these practices help to avoid or detect errors at an early stage of development, which in turn frees capacity in the development process that would otherwise be used for refactoring, bug fixing, or code revision. The free capacity, in turn, allows project members to develop more extensive responses to

How Agile Practices Impact Customer Responsiveness and Development Success

customer change requests. We therefore propose:

P2: Development practices supporting self-control and autonomy positively affect software team response extensiveness.

Agile management practices, such as daily stand-up meetings (Schwaber & Beedle, 2002) and small releases (Beck, 1999) specify rules and procedures, which prescribe behaviors and processes that must be followed. They therefore control how a software team reacts to changed requirements and which procedures it follows in making changes during development. For example, they stipulate how to decide on how many of the features in the product backlog to include in the next release. Feature-related decisions are commitments toward the responses to customer requirements changes. By forcing the team to critically match available resources (time and money) with customer requests, commitments become realistic and can therefore be met more readily. A positive effect will be more restrictive and faster responses to change requests. We therefore propose:

P3: Management practices specifying rules and behavioral procedures positively affect software team response efficiency.

As a third kind of agile practice, we examine **agile standards and norms**. During agile development, the responsibility for delivered software code can be enforced using norms such as collective code ownership or coding standards. Since these standards and norms relate to existing code—that is features of the product to be built—they do not directly relate to the team's ability to respond to customer requests extensively or efficiently. Nonetheless, we expect positive impacts on customer satisfaction and technical software functionality, for example, because of enforced responsibility for the delivered code.

We therefore propose a direct impact on information systems development success:

P4: Agile standards and norms enforcing shared rituals and artifacts positively affect information systems development success, but not software team response extensiveness or efficiency.

Together, these four propositions allow for a meaningful evaluation of agile practices in terms of types and effects. This is the core thesis of our model as shown in Figure 1: The agility of software teams in terms of response extensiveness and response efficiency will impact information systems development success (P1). This has been demonstrated before (e.g., Lee & Xia, 2010; Sabherwal & Chan, 2001); however, now our model additionally proposes that agility of the team can be influenced by the appropriate deployment of specific kinds of agile practices. Management practices such as stand-up meetings or retrospectives and development practices such as pair programming or continuous code integration impact the two orthogonal dimensions of agility—response extensiveness (P2) and response efficiency (P3)—but not both. On the other hand, agile standards and norms will not impact customer responsiveness but will have a direct impact on successful information systems development (P4).

We make one note about our research model. We deliberately constructed it on an abstract, conceptual level, using propositions rather than hypotheses. In turn, our definitions come with an inevitable “openness of meaning” (Kaplan, 1998/1964, pp. 62–79). This means that we developed propositions for evaluation about *kinds of* development practices, management practices, and standards and norms in general, rather than precise hypotheses about certain *types of* practices (such as collective code ownership, pair programming, or refactoring) in particular. Our rationale was motivated

methodologically, theoretically, and empirically. Methodologically, measuring and testing hypotheses on all the available agile practices would be a cumbersome effort, one that is better achieved programmatically through replication (Berthon, Pitt, Ewing, & Carr, 2002) rather than in one study. Theoretically, describing the underlying rationale and justificatory logic of orchestration for multiple practices would be hard to achieve, whereas on a more abstract level (such as the one we propose), we can develop conceptual logic that links different practices to different mechanisms and in turn different outcomes. Empirically, given that we studied agile development in one organization as discussed following, we had to make compromises between extensiveness and measurement of study and the constraints imposed on us by both the partner organization regarding survey length and procedures as well as the particular practices employed in the field situation.

Research Method

Field Study Design

To evaluate our research model, we were faced with the choice of multiple case studies, a cross-sectional survey across information systems development projects in multiple organizations, and an in-depth field study of agile development in one organization. We decided on the latter. Our motivation was threefold: First, the literature reports multiple case studies and cross-sectional surveys on agile information systems development (Hummel et al., 2013) but few field studies, which allowed us to generate contrast in empirical insights. Second, rather than collect data on the entire population under study as in a cross-sectional design (that is, all teams, whether they use agile approaches or not), we wished to examine only individuals with the specific characteristic of using agile information systems development. This also offers the potential of identifying potential confounding factors via the control variables. Finally,

the field study design offered us the potential to confront our hypotheses with a real world situation through the detailed study of a single organization. In turn, this move maximized ecological validity, which we believe is of particular importance given the industry relevance and popularity of agile information systems development.

In our field study, we surveyed project personnel working in agile software teams at a large international retailer that operates roughly 1,000 stores and employs more than 180,000 people. The large majority of stores in the retail organization offer food, liquor, and general merchandise. The information technology division in the organization is responsible for operations and the back office. It distinguishes between two types of information systems development projects. Large projects are managed in a program called 'strategic delivery.' Projects in this program are run in a more traditional, waterfall-type approach, whereas smaller projects are managed in a program called 'rapid delivery,' which follows agile approaches.

The information technology division delivers two to three large projects per year following traditional information systems development approaches. Other, often smaller projects are managed using agile approaches by the rapid delivery team; this team delivers more than 10 small projects in a year. The main goal of introducing agile approaches in the rapid delivery program was to increase the delivery turnaround time for projects.

In this division, the term agile information systems development is not used as a reference to a specific agile methodology; rather it is understood as the iterative delivery of working software and the frequent involvement of the customer throughout the project. Overall, the employees of the division consider their own maturity in agile as low to medium. Agile practices were introduced in 2011 and have not been implemented in all projects. Challenges

concerning the introduction and use of agile information systems development mainly concerned the project organization and management structure, which is still organized in a traditional way so that the employment of agile practices is sometimes hindered by the demand for delivering detailed documents such as business plans and requirements specifications. The goal of the division is to employ agile practices in more projects in the foreseeable future.

Procedures

We studied the rapid delivery program in the retail organization. Our study spanned a period of eight months and included three points of measurement to be able to collect data near the beginning, middle, and end of agile development projects. This procedure was similar to that followed by Maruping, Venkatesh, and Agarwal (2009).

At the first point of measurement, we distributed an online questionnaire to the software development department, in which we asked for demographics and captured control variables such as software team autonomy and diversity (Lee & Xia, 2010). The second questionnaire was distributed three months later to the people who completed the first questionnaire. In the second questionnaire, we captured responses about a varied set of agile practices in use during the software development projects the respondents were currently involved in. Lastly, we distributed the third questionnaire three months after the second questionnaire to all those who had completed the first and second questionnaires. In the third questionnaire, we captured responses about the outcomes of the projects they were involved in, as well as the perceptions about team agility and cohesion during the course of the project. Based on the discussions with the partner organization, we assumed that participants of our study who work in the rapid delivery program should have delivered, on average, two projects in the time span between the second and the third questionnaires.

This allowed us to gather reflective data on past and current projects and their outcomes, in turn increasing the validity of the self-report measures for our dependent variable.

The limitation that ensued was that we could not control for person-project relations. Still, this approach allowed us to assume that respondents are well aware of both (1) the agile practices they actually used and (2) the actual outcomes achieved in any of their past projects in terms of how responses to customer requests were handled and which development outcomes were achieved. Additionally, this procedure allowed us (3) to align the causal logic of our arguments (that practices influence agility and *in turn* information systems development success) with the temporal mechanisms of data collection (in the sense that we first captured data on practices in use and later data on outcomes).

During each round, we sent out reminders to increase the response rate. Participants who filled out all three questionnaires could win registration to an agile conference and an iPad.

The online surveys were created using the open source tool, LimeSurvey. Management provided us with a list of the 360 employees involved in the information technology division and relevant product and customer departments, including names and email addresses. The invitations for the survey participation were sent out via emails, which contained a personalized link for each employee. The personalized links allowed us to track the responses of the same individual over the three measurement periods.

The level of our research model is the team level but operationally we collected data from individuals. This procedure was carried out in similar studies with the intent to understand the behavior of a team by examining data from individual team members (e.g., Guinan & Coopridge, 1998; Keil, Rai, & Liu, 2012). Although we cannot control for all subtle variations within

How Agile Practices Impact Customer Responsiveness and Development Success

each team, we ensured that we examined data only from those respondents that were team members across all points of measurement.

Measurement

In developing measurements to capture data about our propositions, one key decision for us was which *type of practice* to capture to reflect on the *kind of practice* our research model and propositions related to. The reason for this is because, as indicated in Table 1, multiple types of practices could relate to the three kinds of practices we distinguish, viz., management or development practices or agile standards and norms. In working with our partner organization, however, it became clear (1) that not all practices available in textbooks were in use, and (2) that we could not administer an extensive survey that would allow us to measure all potential practices.

In deciding on our measurement strategy, we therefore followed the following criteria in selecting specific agile practices:

1. **Coverage:** We sought to examine practices across a range of popular methodologies, such as Scrum and Extreme Programming.
2. **Measurement:** We sought to focus on practices with existing scales in the academic literature rather than having to invent new measures.
3. **Relevance:** The practices we focus on are popular in use (VersionOne, 2015).
4. **Access:** We selected practices in use at the partner organization. For example, practices associated with Crystal (Cockburn, 2001) were not in use.

As a result, we selected the management practice “stand-up meeting” (STM) and used an existing reflective scale originally proposed by So and Scholl (2009) for measurement. We selected the development practice “pair programming” (PP) and the standard “collective code ownership” (CCO), for

both of which reflective scales existed (Maruping, Venkatesh, & Agarwal 2009; Maruping, Zhang, & Venkatesh 2009).

Agility was measured with the two scales for software team response extensiveness (EXT) and efficiency (EFF) provided by Lee and Xia (2010).

Information systems development success was operationalized as a second-order reflective construct with three dimensions. First, we measured the dimension “software functionality” (SF) reflectively as proposed by Lee and Xia (2010). Second, since we were not able to collect objective success measures for the on-time and on-budget completion of the projects, we decided to combine those two dimensions in one perceptive measure called “process performance” (PPF), as proposed by Wallace et al. (2004). Third, we included “customer satisfaction” (CSF) as a dimension of information systems development success because satisfying the customer is one of the fundamental principles of agile information systems development, as advocated in the *Agile Manifesto* (Fowler & Highsmith, 2001). Customer satisfaction was measured by adapting the reflective semantic differential scale by Bhattacharjee (2001).

We make two notes about this operationalization of information systems development success. First, it differs on two counts according to Lee and Xia (2010): (1) we operationalized two of the success dimensions differently; and (2), because we had no access to objective data, we used only reflective perceptual measures for each first-order construct. Second, we did not measure information systems development success across a sum of projects; rather, each individual rated the most recent project he or she was involved in. This is an accepted and often used way of measuring (Keil, Mann, & Rai, 2000; Nidumolu, 1995; Wallace, Keil, & Rai, 2004) and one we could agree on with our partner organization.

Finally, we decided to also include the scales for team autonomy (AUTO)

and team diversity (DIV) from Lee and Xia (2010), as control variables. We decided to include these measures to allow us to compare our model with their model and a fully integrated model, as part of a post-hoc analysis of our results.

Appendix A lists all measures. Items were measured using 7-point Likert scales, ranging from “Strongly Disagree” to “Strongly Agree,” with three exceptions: software team response extensiveness (EXT) was measured as ordinal percentage (from 0% to 100%), which was transformed into a 6-point scale. Software team response efficiency (EFF) was measured on a 7-point Likert scale, ranging from “Very little” to “Very much.” Like Lee and Xia (2010), we reversed the software team response efficiency item scores for data analysis such that higher scores indicate higher response efficiency. Customer satisfaction (CSF) was measured on a 7-point semantic differential scale. We varied some scale labels in order to reduce the influence of the measurement instrument on the answers of the participants (Fowler, 2001). We worded items in the first two questionnaires such that they referred to the current development project of the participants (as proposed by Hsu, Lin, Cheng, & Linden, 2012). The items of the last questionnaire referred to the completion of said development project so that we were able to capture the project outcome.

Data Analysis

Descriptive Statistics

Of the 102 participants who provided complete responses to the survey at measurement point 1, 79 participants responded to the second survey and 71 responded to all three surveys, equaling an effective response rate of 19.7% and an overall attrition rate of 30.4%. Both the overall attrition rate and pattern of attrition of 22.6% (between points 1 and 2) and 10.1% (between points 2 and 3) characterize our sample as consisting of “high lurkers”

(Lugtig, 2014) with declining yet stable response patterns.

The average age of the participants was 43 years and the average reported team size was 38. All main roles typically associated with agile information systems development—namely developers, product owners, and Scrum-Masters—were well represented in our sample. The product owner role is the most frequently incorporated role of the participants (22.5%), which is similar to the reported occurrence of this role in agile practitioner surveys (e.g., Kim, 2013, p. 3). Still, because the share of product owners in our sample appeared high, we decided to check for response bias between product owners and all other roles. To that end, we compared latent variable scores for all constructs using an independent sample t-test. There were no significant differences between product owners and other respondents for any construct except for software functionality ($t = 2.183$, $p = 0.032$), which was evaluated higher by product owners. This seems reasonable given that the role of the product owner is to assume content authority; that is, as the key stakeholder the product owner needs to have a vision of what he or she wishes to build and convey that vision to the information systems development team. Thus we suggest that no substantial response bias is present.

As expected, the majority of participants were included in information systems development projects for the retail domain and most participants were highly experienced in information systems development. Table 2 summarizes the selected descriptive data about participants and projects. The high number of unspecified answers likely resulted from participants being allowed not to answer demographic questions, which was a requirement imposed on us by the partner organization in order to decrease survey fatigue by their employees.

We evaluated the possibility of common method bias through three tests.

Individual Variables	Results	Project Variables	Results
Experience in information systems development		Team size	
< 1 Year	0.0%	5 or less	9.9%
< 1–2 Years	4.2%	6 to 10	21.1%
< 2–5 Years	8.5%	11 to 20	12.7%
< 5–10 Years	21.1%	21 to 50	12.7%
> 10 Years	31.0%	More than 50	14.1%
Not specified	35.2%	Not specified	29.6%
Experience in agile information systems development		Project domain	
< 1 Year	0.0%	Retail	66.2%
< 1–2 Years	22.5%	Logistics	2.8%
< 2–5 Years	18.3%	Not specified	29.6%
< 5–10 Years	4.2%		
> 10 Years	2.8%		
Not specified	52.1%		
Project role			
Product owner	22.5%		
Project manager	11.3%		
ScrumMaster	9.9%		
Quality assurance/testing	9.9%		
Developer	8.5%		
Information technology manager	8.5%		
Architect	5.6%		
Agile coach	4.2%		
Other	21.1%		

Table 2: Descriptive statistics of final survey sample.

First, we ran Harman's One Factor Test. The results of the principal component analysis show that only 24.5% of the variance is explained by one single factor, providing initial evidence that common method bias is not an issue in our data (Podsakoff, MacKenzie, Lee, & Podsakoff, 2003). Second, we used the Marker-Variable Technique (Malhotra, Kim, & Patil, 2006). The results show that significant paths in the model without the inclusion of the marker variable stay significant when the marker variable is included in the model, indicating that common method bias is not a serious concern. Third, we separated measurement by distributing three questionnaires at different points in time, thereby reducing the likelihood of

common method bias (Podsakoff et al., 2003).

Measurement Validation

We analyzed the data using partial least squares analysis with SmartPLS (Ringle, Wende, & Will, 2005). We evaluated established guidelines (Gefen, Rigdon, & Straub, 2011; Goodhue, Lewis, & Thompson, 2012; Hair, Hult, Ringle, & Sarstedt, 2014; Ringle, Sarstedt, & Straub, 2012) in order to decide whether to use partial least squares or covariance-based structural equation modeling. Partial least squares modeling is appropriate for analyzing our data because we needed to explore different variants of our research model in order to appropriately evaluate our

How Agile Practices Impact Customer Responsiveness and Development Success

Construct	Cronbach's Alpha	Composite Reliability	AVE	Mean	SD
Collective Code Ownership (CCO)	0.82	0.87	0.57	4.16	1.71
Customer Satisfaction (CSF)	0.88	0.92	0.74	5.03	1.18
Software Team Response Extensiveness (EXT)	0.92	0.94	0.73	3.32	1.51
Software Team Response Efficiency (EFF)	0.89	0.92	0.65	3.46	1.66
Pair Programming (PP)	0.99	0.99	0.98	2.57	1.79
Process Performance (PPF)	0.94	0.97	0.94	4.23	1.94
Software Functionality (SF)	0.94	0.96	0.85	5.62	1.20
Stand-up Meeting (STM)	0.96	0.97	0.88	4.04	2.29

Note: AVE = Average Variance Extracted; SD = Standard Deviation

Table 3: Construct statistics.

propositions, for which an exploratory approach to data analysis is preferred. Also, partial least squares modeling is especially suited for evaluating research models with hierarchical constructs, which was the case in our setting (Wetzels, Odekerken-Schröder, & Van Oppen, 2009).

We modeled all scale items as reflective indicators of their theorized latent construct. The measurement validation also includes the first-order constructs customer satisfaction, process performance, and software functionality, which serve as the basis for the structural evaluation of the second-order

construct information systems development success.

Table 3 summarizes the reliability, average variance extracted, mean, and standard deviation of the latent variables. Table 4 summarizes construct correlations and the evaluation of the Fornell-Larcker criterion. Relevant thresholds for reliability and discriminant validity are met (Fornell & Larcker, 1981; MacKenzie, Podsakoff, & Podsakoff, 2011; Straub, Boudreau, & Gefen, 2004). Table 5 shows the loadings and cross-loadings of the latent variables. Loadings on the designated variables were higher than 0.7, except

for items CCO1 and CCO3, which were just below this threshold (Hair et al., 2014). Loadings on the intended latent variable were also well above the cross-loadings on other variables (Straub et al., 2004). We conclude that indicator, convergent, and discriminant validity is sufficiently present in our data.

Finally, we examined the hierarchical construct of information systems development success, which we modeled as a reflective first-order, reflective second-order construct. ISD success is manifested by the dimensions of customer satisfaction, process performance, and software functionality, because for an information systems development project to be successful, high values in all those three dimensions are needed. We followed guidelines for evaluating hierarchical constructs in partial least squares modeling (Wright, Campbell, Thatcher, & Roberts, 2012): First, we evaluated the measurement model including only the first-order constructs in order to ensure scale validation. Due to the uneven number of indicators of the first-order constructs, we used the standardized latent variable scores of each dimension as indicators of the second-order construct in order to evaluate the structural model and the explained variance.

Structural Model Evaluation

We estimated a structural model with the view to evaluate the propositions in our conceptual model. To be faithful in our evaluation of the propositions, in estimating the structural model, we included all constructs from the research model and included associations between:

1. the development practice (pair programming), management practice (stand-up meetings), and standards and norms (collective code ownership) to both software team response extensiveness and efficiency;
2. the development practice (pair programming), management practice (stand-up meetings), and standards

Construct	CCO	CSF	EFF	EXT	PP	PPF	SF	STM
Collective code ownership (CCO)	0.75							
Customer satisfaction (CSF)	0.38	0.86						
Software team response efficiency (EFF)	0.10	0.14	0.81					
Software team response extensiveness (EXT)	0.02	0.32	−0.39	0.85				
Pair programming (PP)	0.04	0.03	−0.05	0.23	0.99			
Process performance (PPF)	0.20	0.45	0.23	0.08	0.18	0.97		
Software functionality (SF)	0.34	0.75	0.24	0.12	0.02	0.54	0.92	
Stand-up meeting (STM)	0.48	0.11	−0.01	−0.13	0.28	0.13	0.4	0.94

Note: Diagonal elements represent the square root of the average variance extracted. Off-diagonal elements are the correlations among latent constructs.

Table 4: Construct correlations and Fornell-Larcker criterion analysis.

Construct	Item	CCO	CSF	EFF	EXT	PP	PPF	SF	STM
Collective code ownership (CCO)	CCO1	0.620	0.156	0.155	−0.030	−0.113	0.045	0.083	0.321
	CCO2	0.712	0.208	−0.055	0.067	0.041	0.071	0.222	0.251
	CCO3	0.683	0.182	0.212	−0.030	−0.001	0.133	0.123	0.315
	CCO4	0.832	0.319	−0.100	0.133	0.091	0.068	0.270	0.421
	CCO5	0.872	0.408	0.170	−0.046	0.058	0.283	0.407	0.438
Customer satisfaction (CSF)	CSF1	0.351	0.859	0.127	0.244	0.015	0.524	0.813	0.149
	CSF2	0.210	0.813	0.076	0.291	−0.145	0.381	0.608	0.040
	CSF3	0.358	0.899	0.123	0.261	0.096	0.371	0.610	0.069
	CSF4	0.357	0.858	0.138	0.296	0.114	0.287	0.542	0.119
Software team response efficiency (EFF)	EFF1	0.030	0.128	0.784	−0.298	0.128	0.195	0.154	0.042
	EFF2	0.026	0.044	0.831	−0.425	−0.168	0.224	0.169	0.029
	EFF3	0.121	0.136	0.912	−0.356	−0.100	0.182	0.209	−0.004
	EFF4	0.022	−0.018	0.676	−0.176	0.113	0.232	0.092	−0.024
	EFF5	0.145	0.198	0.886	−0.352	−0.034	0.195	0.254	0.026
	EFF6	0.128	0.124	0.725	−0.212	−0.103	0.106	0.229	−0.147
Software team response extensiveness (EXT)	EXT1	−0.020	0.111	−0.285	0.810	0.093	0.078	0.010	−0.319
	EXT2	−0.001	0.195	−0.344	0.874	0.295	0.030	0.058	−0.122
	EXT3	0.062	0.267	−0.376	0.922	0.224	0.087	0.106	−0.083
	EXT4	0.118	0.352	−0.242	0.793	0.129	0.048	0.220	−0.024
	EXT5	−0.021	0.298	−0.370	0.880	0.215	0.068	0.109	−0.090
	EXT6	−0.038	0.377	−0.342	0.827	0.189	0.072	0.121	−0.059
Pair programming (PP)	PP1	0.052	0.031	−0.012	0.244	0.984	0.208	0.047	0.255
	PP2	0.037	0.043	−0.077	0.225	0.992	0.144	0.015	0.281
	PP3	0.040	0.017	−0.056	0.207	0.988	0.178	−0.001	0.289
Process performance (PPF)	PPF1	0.189	0.421	0.214	0.117	0.194	0.973	0.519	0.124
	PPF2	0.191	0.461	0.235	0.023	0.154	0.966	0.531	0.129
Software functionality (SF)	SF1	0.403	0.704	0.162	0.158	0.013	0.504	0.939	0.187
	SF2	0.327	0.732	0.185	0.156	0.044	0.516	0.951	0.093
	SF3	0.269	0.745	0.232	0.145	−0.037	0.516	0.942	0.063
	SF4	0.247	0.561	0.313	−0.033	0.070	0.455	0.847	0.070
Stand-up meeting (STM)	STM1	0.393	0.018	0.085	−0.164	0.215	0.077	−0.007	0.903
	STM2	0.444	0.132	−0.041	−0.121	0.325	0.160	0.156	0.955
	STM3	0.456	0.093	−0.033	−0.142	0.251	0.094	0.087	0.939
	STM4	0.480	0.133	−0.006	−0.079	0.219	0.128	0.136	0.950

Table 5: Item cross-loadings.

and norms (collective code ownership), and both software team response extensiveness and efficiency to the reflective second order construct software development success; and

3. between software team response extensiveness and efficiency as proposed by (Lee & Xia, 2010).

The structural model results are shown in Figure 2. The model explains

30% of the variance in information systems development success, and 11% and 19% of software team response extensiveness and efficiency, respectively.

Concerning the proposed relationships in the model, our data show that, as

How Agile Practices Impact Customer Responsiveness and Development Success

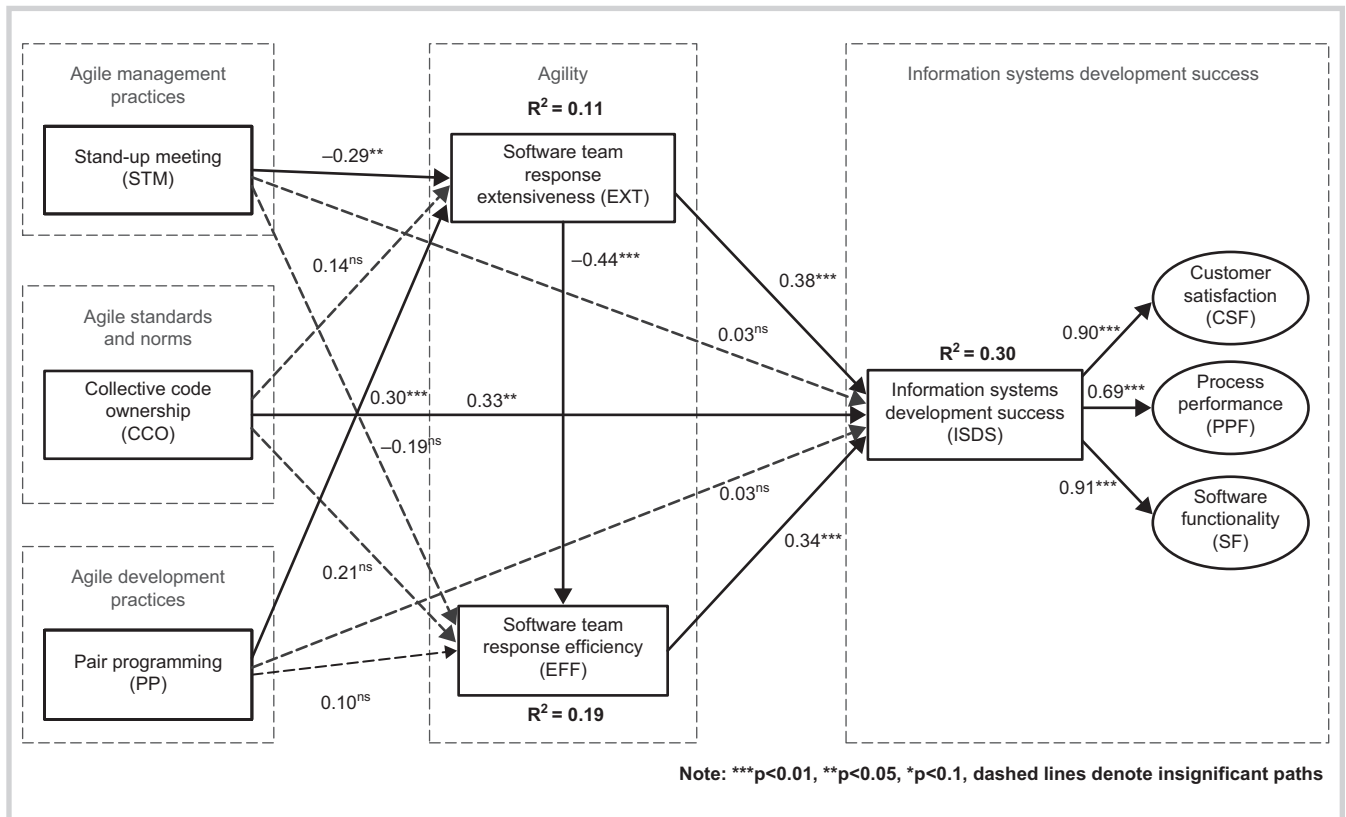


Figure 2: Structural model results.

proposed by Lee and Xia (2010), software team response extensiveness and efficiency are both significantly associated with ISD success ($\beta = 0.38$, $p < 0.01$ for software team response extensiveness and $\beta = 0.34$, $p < 0.01$ for software team response efficiency). The management practice stand-up meetings was negatively associated with software team response extensiveness ($\beta = -0.29$, $p < 0.05$) and not significantly related to either software team response efficiency or information systems development success. Conversely, the development practice pair programming was *positively* associated with software team response extensiveness ($\beta = 0.30$, $p < 0.01$) and not significantly related to either software team response efficiency or information systems development success. Finally, the standards and norms collective code ownership was positively related to information systems development success ($\beta = 0.33$, $p < 0.05$) but not significantly

associated with either software team response extensiveness or efficiency.

Table 6 reports effect sizes (f^2) and variance inflation factors (VIF) for all exogenous constructs. All variance inflation factors values are well below the threshold of 5.0, indicating no

multicollinearity issues for our structural model results. Concerning effect sizes: the effects from software team response extensiveness on software team response efficiency and software team response extensiveness on information systems development success are

Variable	EXT	EFF	Information Systems Development Success
CCO	0.016 (1.303)	0.041 (1.323)	0.111 (1.377)
PP	0.094 (1.095)	0.009 (1.197)	0.001 (1.209)
STM	0.066 (1.407)	0.029 (1.499)	0.001 (1.543)
EXT		0.211 (1.124)	0.150 (1.361)
EFF			0.133 (1.235)

Table 6: Effect sizes (variance inflation factors) for exogenous constructs.

medium, whereas the effects from pair programming and stand-up meetings on software team response efficiency and from collective code ownership on information systems development success are small. The effect of software team response efficiency on information systems development success, too, is small (but at the upper range).

Post-hoc Analyses

We performed several post-hoc analyses to examine the robustness of our results. First, we decomposed information systems development success into its three constituent dimensions and re-estimated the structural model. Appendix B provides details of this analysis. The results are comparable with those illustrated in Figure 2.

Second, we compared our model with two alternative conceptualizations: (1) the model by Lee and Xia (2010), which we replicated with our data; and (2) an integrated model that adds to our model the team characteristics autonomy and diversity from Lee and Xia (2010). Appendix C provides these details; again, we find that our results are robust against these additional analyses.

Third, we carried out a mediation analysis to better understand the results we obtained about the relation between software team response extensiveness and efficiency to information systems development success. Specifically, we were interested in ascertaining whether and how the construct software team response efficiency mediates the relation between the constructs software team response extensiveness and information systems development success. Tables 7 and 8 summarize the results from this analysis. A variable functions as a mediator when it meets three conditions (Baron & Kenny, 1986):

1. Variations in the independent variable (software team response extensiveness) account significantly for variations in the presumed mediator (software team response efficiency), which is given in our model.

	CCO	EFF	EXT	ISDS	PP	STM
CCO		0.209	0.135	0.326		
EFF				−0.337		
EXT		−0.438		0.376		
ISDS						
PP		0.095	0.302	0.026		
STM		−0.189	−0.287	0.032		

Table 7: Path coefficients in model with mediator (software team response efficiency) in the model.

2. Variations in the mediator account significantly for the variations in the dependent variable (information systems development success), which is given in our model.
3. When paths between the independent variable (software team response extensiveness) and mediator (software team response efficiency) and between the mediator (software team response efficiency) and dependent variable (information systems development success) are controlled, a previously significant relation between the independent (software team response extensiveness) and dependent variables (information systems development success) changes its value significantly. This is the case for our model, because we have a change from the model with mediator in the path coefficient for software team response extensiveness $\rightarrow$ information systems development success, from 0.376 down to 0.240 in the model without the mediator (see bold and underscored numbers in Tables 7 and 8).

We interpret these results as a strong indicator for mediation. In other words,

the mediator (software team response efficiency) explains a large share of the effect from software team response extensiveness on information systems development success. Furthermore, the variance accounted for statistic (−1.598) for the construct software team response efficiency is negative, which classifies the type of mediation as a full mediation effect (Hair et al., 2014, p. 225).

Discussion

Summary of Findings

Table 9 summarizes the insights gained from our propositions and summarizes our interpretations; in turn, we discuss the notable findings.

Regarding Proposition P1, our research confirms that agility is an important predictor of information systems development success and thereby corroborates the findings by Xia and Lee (2010); like them we found that extensive responses to changing customer requirements induces lowered response efficiency, suggesting that extensive responses require additional response efforts, which decreases response efficiency. Moreover, same as Xia and Lee (2010), we also found that software team

	CCO	EXT	ISDS	PP	STM
CCO		0.151	0.401		
EXT			0.240		
ISDS					
PP		0.300	0.049		
STM		−0.294	−0.031		

Table 8: Path coefficients in model without mediator (software team response efficiency) in the model.

How Agile Practices Impact Customer Responsiveness and Development Success

No	Relevant Empirical Results	Interpretation
P1	Software team response extensiveness is significantly positively related to information systems development success. Software team response efficiency is significantly positively related to information systems development success.	Agility is an important predictor of information systems development success, both in terms of responding extensively and efficiently to customer requirements. These results corroborate those of Lee and Xia (2010).
P2	Pair programming is significantly positively related to software team response extensiveness; it is not significantly related to customer response efficiency or information systems development success.	Pair programming in agile development is important to ensure good quality in software development, in turn allowing the team to respond broadly and comprehensively to inquiries.
P3	Stand-up meetings are significantly negatively related to software team response extensiveness. They are not significantly related to software team response efficiency or information systems development success.	Stand-up meetings reduce a team's ability to respond broadly and comprehensively. Their influence on efficiency is mediated through response extensiveness.
P4	Collective code ownership is significantly positively related to information systems development success but not to agility.	Collective code ownership does not affect a team's ability to respond to customer requirements changes.

Table 9: Findings about propositions.

response efficiency contributes to process performance, customer satisfaction, and software functionality, which is particularly noteworthy because we used different scales to measure information systems development success. Xia and Lee (2010) report positive relations from additional efforts for response efficiency to on-time completion, on-budget completion, and software functionality, and we now find that similar positive flow on effects accrue for process performance and customer satisfaction.

Regarding propositions P2 through P4, our data suggest that different kinds of agile practices impact agility, and in turn ISD success, in different ways. We find that standards and norms (viz., collective code ownership) are important *direct antecedents* of successful information systems development but they do not influence how software teams respond to customer requirements. This is because standards and norms enforce a group behavioral culture, which is characterized by group members agreeing on ways of working. Yet, they do not influence or prescribe outcomes of such shared routines and rituals, and thus do not impact

whether or not a team responds effectively or efficiently. Development practices (viz., pair programming), on the other hand, directly impact the range of response of a software team to customer requirements because they provide local best practices that can optimize the output created during development work. Management practices, finally, inhibit the range of responses, and in doing so indirectly improve response efficiency. As for the specific practice of stand-up meetings, this may depend on whether the customer or product owner is part of the daily meetings. If not, team meetings take time away from discussions for clarifying requirements with customers or product owners, speeding up the response process.

Implications for Research

Our research provides a rationale for how and why selected used agile practices relate to successful information systems development projects. In doing so, we believe our work extends on the previous studies on team characteristics as antecedents to agility. We extend this research with a focus on

practices. Software teams enact specific practices; in so far, both groups (their characteristics) and their behaviors (the practices they use) are important and inter-dependent components of agility. At the same time, our findings indicate that the existing models are not perfect, with overall medium variance explained and not being completely replicable; other theories might be needed to understand and provide support for the effects of agility in information systems development.

Abrahamsson, Conboy, and Xiaofeng (2009) stressed the need to understand what constitutes agility. While on the surface we replicate the importance of software team response effectiveness and software team response efficiency to agility, we also go a step further than Xia and Lee (2010). They considered software team response effectiveness and efficiency to be similar or nearly identical dimensions of agility, whereas we show that they are affected differently by agile practices. This finding leads to new research questions. For example, using Conboy's (2009) taxonomy of agility, only one of his categories—rapid change—is covered by software team response efficiency and extensiveness. This suggests that future research needs to address the other categories of Conboy's (2009) taxonomy—for example, economy, quality, simplicity, or learning from change—to examine which other constituent elements form the agility of software teams.

A second line of research could expand on our empirical design, for example, by studying compare software team responsiveness and efficiency *across projects*, and identify improvements or decline in them over time. This includes studies under which conditions and goals with which teams, customers, and practices; and what detrimental or advantageous effects on software response effectiveness and efficiency are obtainable. A further step would then link them to information systems development success. This could be expanded to a contingency model of

agile information systems development and to a configuration inventory, which specifies a “fit” between specific information systems development situations and corresponding agile practices. This also includes a comparison of using different agile practices in different settings (e.g., such as co-located versus distributed, Sarker & Sarker, 2009).

There are also interesting implications for theory that follow from our proposed categorization of agile practices into three distinct sets: management practices, development practices, and standards and norms. Our empirical data suggest that practices are indeed a core component driving agility in information systems development and may be equally, or even more, essential and vital than teams and their members’ characteristics (which is backed by our comparisons of different models in Appendix C): Good developers or good teams may well be necessary for agility and information systems development success, but good teams using suitable practices are even better. This finding can now provide the impetus for several studies: for example, a future study might elaborate on the fit between situational characteristics and suitability of agile practices. The important part here may be that agile information systems development teams take charge of their own ways of working and select the practices that benefit them the most. Another direction for future work may stem from a need for further theorizing: The empirical differences we uncovered may require new theories that explain the effect of different practices. For example, models of group behavior or team work mechanisms (Ilgen, Hollenbeck, Johnson, & Jundt, 2005; Kozlowski & Ilgen, 2006) might be suitable for investigating the effect of management practices rather than development practice practices, whereas theories building on models of self-regulation and self-organization (Karhatsu, Ikonen, Kettunen, Fagerholm, & Abrahamsson, 2010; Varela, 1984) may be more appropriate for the latter. Similarly,

we need to explore which facets (Sarker, Munson, Sarker, & Chakraborty, 2009), components (Conboy, 2009), or patterns (Baskerville, Pries-Heje, & Madsen, 2011) of agility are affected by which practice sets.

We also showed that different practices provide different effects on agility and the outcomes of information systems development. Our findings suggest that management and development practice practices have divergent but not necessarily opposite effects on agility, and they highlight the core roles of standards and norms to successful outcomes. Future research should investigate the effect of practices other than the ones we studied, both in isolation and in combined use settings. For example, stand-up meetings should be examined in combination with other practices such as having an onsite customer.

Implications for Agile Development Projects in Particular

For practitioners, our findings help to take the first steps in answering an important question: Which agile practices from which available methodologies should be employed in information systems development projects? Our results suggest that the answer is contingent on the goals: If the goal is to directly improve information systems development success without caring about agile behavior and without any changes in team behavior or composition, standards and norms such as collective code ownership can help to generate successful outcomes without affecting software team responsiveness at all. By contrast, if long-term changes are possible, agile management practices and agile development practices provide an ability to direct teams’ abilities to respond to customers efficiently or effectively, thereby improving agility, which in turn, provides management interventions to positively affect information systems development success. In addition, the choice of agile practices may be more salient for practitioners than attempting to “configure” their teams.

Our results on the impact of software team response efficiency and software team response extensiveness on information systems development success reiterate further that focusing only on quick and rapid changes may not yield desirable results for agile projects, which reinforces the adage that agile development should not necessarily mean fast development. When software teams focus on expanding the range and comprehensiveness of their responses, their ability to respond as quickly as possible diminishes, but the implications for success increase. Information systems development team members need to be incentivized to not just respond as quickly as possible, but to respond reasonably quickly *and* with enough detail and understanding of the situation, domain, and problem at hand.

A final implication here is that organizations should prioritize software team response extensiveness through practices such as pair programming, and closely monitor their use of management practices, such as daily stand-ups, so as not to reduce their response extensiveness.

Implications for Project Management in General

As was the case in our study, much of the research on agile projects focuses on information systems development and software development initiatives or industries (e.g., Qumer & Henderson-Sellers, 2008; Wells, 2012). Still, there is also evidence to suggest that agile methods can be of benefit to project management in general (Conforto, Salum, Amaral, da Silva, & de Almeida, 2014). For instance, startup companies or venture companies often use agile practices to structure daily work processes besides software development (Sutherland & Altman, 2009; 2010). This also leads to combining agile practices from Scrum with lean management practices from Kanban (Wang et al., 2012), and many practices of agile information systems development are similar to their counterparts in lean operations management, which are increasingly also adopted for

How Agile Practices Impact Customer Responsiveness and Development Success

project management. Moreover, agile information systems development methods such as Scrum have their roots in new product (non-software) development (Takeuchi & Nonaka, 1986), and are not necessarily specific for the software industry. Our findings should therefore be transferable, at least to some degree, to other (non-software) projects in general.

For example, the use of standards and norms is advisable in every kind of project. Management practices, such as stand-up meetings, can be employed independently of whether or not the product is software. Development practices, such as pair programming, can be translated to a general project practice to work in pairs on complex tasks and problems (e.g., creative tasks such as ideation). However, not every kind of project in any situation or domain aims at team responsiveness (e.g., initiatives in the military sector in other kinds of mission-critical projects). Therefore, the effects of adopting practices from information systems development, such as stand-up meetings, to other kinds of projects with other goals need to be evaluated carefully. This challenge, in turn, may open up directions for future research on the use of agile practices in the management of projects in general (e.g., studies investigating the diffusion and adoption of agile practices in more general project settings).

Limitations

Our research has several limitations. First, we were not able to conduct our study on the team level; instead, we collected data by individuals about the teams and projects they are involved in. While this approach is common (Keil et al., 2000; Nidumolu, 1995; Wallace et al., 2004), we cannot use it to evaluate the relationships between respondents and projects, or respondents and teams. Therefore, our findings about information systems development success should not be interpreted as the emergent properties of information systems development projects or software teams, rather as the reported experiences of agile practitioners involved in information systems development projects and

contributing to software teams. The upside of this approach is that we can collect data on experiences accrued from more than one project, as done, for example, by Lee and Xia (2010). Still, more research is encouraged to increase the validity of our results by replicating our study on the team level rather than the individual level.

Second, due to the restrictions of our field study regarding survey length, we operationalized agile management practices, development practices, and standards and norms only with one construct each. In turn, the interpretations of our findings in relation to the propositions about the kinds of practices (development versus management practices versus standards) are empirically bounded by the types of practices we collected data on (pair programming, stand-up meetings, and collective code ownership). Interpretations of our results thus need to consider these boundary conditions. Ideally, future studies will replicate our work and consider and measure other types of practices indicative of the kinds of practices we theorized about. For instance, other popular management practices include retrospectives or small releases. Popular development practices include refactoring or continuous code integration. Finally, there are other agile norms such as coding standards. Williams (2012) provides an overview of different practices and their discussion in the literature. We tried to alleviate a potential lack of external validity by examining popular practices across a wide range of methodologies rather than just focusing on one specific methodology (e.g., Maruping, Venkatesh, & Agarwal, 2009; Moe, Dingsøyr, & Dybå, 2010) or just one practice (e.g., Balijepally et al., 2009; Cockburn & Williams, 2001).

Third, we were only able to collect perceptual measures, which notably impacted our conceptualization of information systems development success, and as discussed, deviated somewhat from that of Lee and Xia (2010). This also allowed us, however, to corroborate the existing theory by showing it remains

robust against variations in measurement of the dependent variable. Our approach to data collection is also susceptible to common method bias; yet we conducted several tests and found no indication that systematic bias is present. Fourth, we collected data from the agile development efforts from one large organization. Our findings and their interpretations are thus bounded to the single industry (retail) and organization (large) that we studied. However, we note that our dataset is also original in that most other studies consider cross-sectional data from efforts across many organizations and heterogeneous teams.

Conclusion

Selecting the right practices from a portfolio of methodologies remains a challenge for most organizations trying to become more agile. With this study we sought to extend our understanding of agile practices use and their effects on successful software development. We showed that different practices impact agile teams in terms of extensive or efficient customer responses in different ways and, ultimately contribute to successful software development in different ways. While this finding is interesting in its own right, it also generates a new question about the relation between the technical and the social in agile development: Is the team (the structure) more important or the methodological choice of practices (the tasks and technology)? The question is not conclusively answered by our research but it serves well as impetus to stimulate further inquiry.

Acknowledgments

We would like to thank the case organization for providing access to their development teams. Dr. Recker's contributions were partially supported by a grant from the Australian Research Council (DP160103407). Dr. Rosenkranz's contributions were supported by a grant from the German Research Foundation (DFG) under record no. RO 3650/8-1. Aside from the first author, author names are ordered alphabetically irrespective of contribution.

References

- Abrahamsson, P., Conboy, K., & Xiaofeng, W. (2009). 'Lots done, more to do.' The current state of agile systems development research. *European Journal of Information Systems*, 281–284.
- Abrahamsson, P., Salo, O., Ronkainen, J., & Warsta, J. (2002). *Agile software development methods: Review and analysis*. Oulu, Finland: VTT Electronics.
- Ågerfalk, P. J., Fitzgerald, B., & Slaughter, S. A. (2009). Flexible and distributed information systems development: State of the art and research challenges. *Information Systems Research*, 20(3), 317–328.
- Balijepally, V., Mahapatra, R., Nerur, S., & Price, K. H. (2009). Are two heads better than one for software development? The productivity paradox of pair programming. *MIS Quarterly*, 33(1), 91–118.
- Barki, H., & Hartwick, J. (2001). Interpersonal conflict and its management in information system development. *MIS Quarterly*, 25(2), 195–228.
- Baron, R. M., & Kenny, D. A. (1986). The moderator-mediator variable distinction in social psychological research: Conceptual, strategic, and statistical considerations. *Journal of Personality and Social Psychology*, 51(6), 1173.
- Baskerville, R., Pries-Heje, J., & Madsen, S. (2011). Post-agility: What follows a decade of agility? *Information & Software Technology*, 53(5), 543–555.
- Beck, K. (1999). *Extreme programming explained: Embrace change*. Boston, MA: Addison-Wesley.
- Berente, N., Hansen, S. W., & Rosenkranz, C. (2015). *Rule formation and change in information systems development: How institutional logics shape ISD practices and processes*. Paper presented at the 48th Hawaii International Conference on System Sciences (HICSS 2015), Kauai, Hawaii.
- Berthon, P., Pitt, L., Ewing, M., & Carr, C. L. (2002). Potential research space in MIS: A framework for envisioning and evaluating research replication, extension and generation. *Information Systems Research*, 13(4), 416–427.
- Bhattacharjee, A. (2001). Understanding information systems continuance: An expectation-confirmation model. *MIS Quarterly*, 25(3), 351–370.
- Boehm, B. W. (2002). Software engineering economics. In M. Broy & E. Denert (Eds.), *Software pioneers: Contributions to software engineering* (pp. 641–686). New York, NY: Springer.
- Bostrom, R. P., Gupta, S., & Thomas, D. M. (2009). A meta-theory for understanding information systems within sociotechnical systems. *Journal of Management Information Systems*, 26(1), 17–47.
- Bourque, P., & Fairley, R. E. (2014). *Guide to the software engineering body of knowledge (SWEBOK): Version 3.0*. Piscataway, NJ: IEEE Computer Society Press.
- Cockburn, A. (2001). *Crystal clear: A human-powered software development methodology for small teams*. Reading, MA: Addison-Wesley.
- Cockburn, A., & Williams, L. (2001). The costs and benefits of pair programming. In G. Succi & M. Marchesi (Eds.), *Extreme programming examined* (pp. 223–243). Boston, MA: Addison-Wesley.
- Cohen, S. G., & Bailey, D. E. (1997). What makes teams work: Group effectiveness research from the shop floor to the executive suite. *Journal of Management*, 23(3), 239–290.
- Conboy, K. (2009). Agility from first principles: Reconstructing the concept of agility in information systems development. *Information Systems Research*, 20(3), 329–354.
- Conforto, E. C., Salum, F., Amaral, D. C., da Silva, S. L., & de Almeida, L. F. M. (2014). Can agile project management be adopted by industries other than software development? *Project Management Journal*, 45(3), 21–34.
- Dingsøyr, T., Nerur, S., Balijepally, V., & Moe, N. B. (2012). A decade of agile methodologies: Towards explaining agile software development. *Journal of Systems and Software*, 85(6), 1213–1221.
- Erickson, J., Lyytinen, K., & Siau, K. (2005). Agile modeling, agile software development, and extreme programming: The state of research. *Journal of Database Management*, 16(4), 88–100.
- Fitzgerald, B. (1997). The use of systems development methodologies in practice: A field study. *Information Systems Journal*, 7(3), 201–212.
- Fitzgerald, B., Hartnett, G., & Conboy, K. (2006). Customising agile methods to software practices at Intel Shannon. *European Journal of Information Systems*, 15(2), 200–213.
- Fornell, C., & Larcker, D. F. (1981). Evaluating structural equations with unobservable variables and measurement error. *Journal of Marketing Research*, 18(1), 39–50.
- Fowler, F. J. (2001). *Survey research methods* (3rd ed.). Thousand Oaks, CA: Sage.
- Fowler, M., & Highsmith, J. (2001). The agile manifesto. *Software Development*, 9(8), 28–32.
- Gefen, D., Rigdon, E. E., & Straub, D. W. (2011). An update and extension to SEM guidelines for administrative and social science research. *MIS Quarterly*, 35(2), iii–xiv.
- Goodhue, D. L., Lewis, W., & Thompson, R. L. (2012). Comparing PLS to regression and LISREL: A response to Marcoulides, Chin, and Saunders. *MIS Quarterly*, 36(3), 703–716.
- Guinan, P. J., & Coopridge, J. G. (1998). Enabling software development team performance during requirements definition: A behavioral versus technical approach. *Information Systems Research*, 9(2), 101–125.
- Hair, J. F., Hult, G. T. M., Ringle, C. M., & Sarstedt, M. (2014). *A primer on partial least squares structural equation modeling (PLS-SEM)*. Thousand Oaks, CA: SAGE Publications.
- Hazzan, O., & Dubinsky, Y. (2003). Bridging cognitive and social chasms in software development using extreme programming. In M. Marchesi & G. Succi (Eds.), *Extreme programming and agile processes in software engineering-XP2002* (Vol. 2675, pp. 47–53). Genova, Italy: Springer.

How Agile Practices Impact Customer Responsiveness and Development Success

- Hsu, J. S.-C., Lin, T.-C., Cheng, K.-T., & Linden, L. P. (2012).** Reducing requirement incorrectness and coping with its negative impact in information system development projects. *Decision Sciences*, 43(5), 929–955.
- Hummel, M., Rosenkranz, C., & Holten, R. (2013).** The role of communication in agile systems development: An analysis of the state of the art. *Business & Information Systems Engineering*, 5(5), 343–355.
- Iivari, J., & Iivari, N. (2011).** The relationship between organizational culture and the deployment of agile methods. *Information and Software Technology*, 53(5), 509–520.
- Ilgen, D. R., Hollenbeck, J. R., Johnson, M., & Jundt, D. (2005).** Teams in organizations: From input-process-output models to IMOI models. *Annual Review of Psychology*, 56(1), 517–543. doi: doi:10.1146/annurev.psych.56.091103.070250
- Jacobson, I., Ng, P.-W., McMahon, P., Spence, I., & Lidman, S. (2012).** The essence of software engineering: The SEMAT kernel. *Queue*, 10(10), 40.
- Kaplan, A. (1998/1964).** *The conduct of inquiry: Methodology for behavioral science*. Piscataway, NJ: Transaction Publishers.
- Karhatsu, H., Ikonen, M., Kettunen, P., Fagerholm, F., & Abrahamsson, P. (2010, 3–5 Oct. 2010).** *Building blocks for self-organizing software development teams a framework model and empirical pilot study*. Paper presented at the International Conference on Software Technology and Engineering, San Juan, Puerto Rico.
- Keil, M., Mann, J., & Rai, A. (2000).** Why software projects escalate: An empirical analysis and test of four theoretical models. *MIS Quarterly*, 24(4), 631–664.
- Keil, M., Rai, A., & Liu, S. (2012).** How user risk and requirements risk moderate the effects of formal and informal control on the process performance of it projects. *European Journal of Information Systems*, 22(6), 650–672.
- Kim, D. (2013).** *The state of Scrum: Benchmarks and guidelines*. Retrieved from [https://www.scrumalliance.org/scrum/media/ScrumAllianceMedia/Files%20](https://www.scrumalliance.org/scrum/media/ScrumAllianceMedia/Files%20and%20PDFs/State%20of%20Scrum/2013-State-of-Scrum-Report_062713_final.pdf)
- Kirsch, L. J. (1996).** The management of complex tasks in organizations: Controlling the systems development process. *Organization Science*, 7(1), 1–21.
- Kozlowski, S. W. J., & Ilgen, D. R. (2006).** Enhancing the effectiveness of work groups and teams. *Psychological Science in the Public Interest*, 7(3), 77–124. doi: 10.1111/j.1529-1006.2006.00030.x
- Lee, G., & Xia, W. (2010).** Toward agile: An integrated analysis of quantitative and qualitative field data on software development agility. *MIS Quarterly*, 34(1), 87–114.
- Lugtig, P. (2014).** Panel attrition: Separating stayers, fast attriters, gradual attriters, and lurkers. *Sociological Methods and Research*, 43(4), 699–723.
- MacCormack, A., Verganti, R., & Iansiti, M. (2001).** Developing products on “Internet Time:” The anatomy of a flexible development process. *Management Science*, 47(1), 133–150.
- MacKenzie, S. B., Podsakoff, P. M., & Podsakoff, N. P. (2011).** Construct measurement and validation procedures in MIS and behavioral research: Integrating new and existing techniques. *MIS Quarterly*, 35(2), 293–334.
- Malhotra, N. K., Kim, S. S., & Patil, A. (2006).** Common method variance in IS research: A comparison of alternative approaches and a reanalysis of past research. *Management Science*, 52(12), 1865–1883.
- Maruping, L. M., Venkatesh, V., & Agarwal, R. (2009).** A control theory perspective on agile methodology use and changing user requirements. *Information Systems Research*, 20(3), 377–399.
- Maruping, L. M., Zhang, X., & Venkatesh, V. (2009).** Role of collective ownership and coding standards in coordinating expertise in software project teams. *European Journal of Information Systems*, 18(4), 355–371.
- McHugh, O., Conboy, K., & Lang, M. (2014).** Agile practices: The impact on trust in software project teams. *IEEE Software*, 29(3), 71–76.
- Moe, N. B., Dingsøyr, T., & Dybå, T. (2010).** A teamwork model for understanding an agile team: A case study of a scrum project. *Information and Software Technology*, 52(5), 480–491.
- Nidumolu, S. R. (1995).** The effect of coordination and uncertainty on software project performance: Residual performance risk as an intervening variable. *Information Systems Research*, 6(3), 191–219.
- Pikkarainen, M., Haikara, J., Salo, O., Abrahamsson, P., & Still, J. (2008).** The impact of agile practices on communication in software development. *Empirical Software Engineering*, 13(3), 303–337.
- Podsakoff, P. M., MacKenzie, S. B., Lee, J.-Y., & Podsakoff, N. P. (2003).** Common method bias in behavioral research: A critical review of the literature and recommended remedies. *Journal of Applied Psychology*, 88(5), 879–903.
- Qumer, A., & Henderson-Sellers, B. (2008).** An evaluation of the degree of agility in six agile methods and its applicability for method engineering. *Information and Software Technology*, 50(4), 280–295.
- Ringle, C. M., Sarstedt, M., & Straub, D. W. (2012).** Editor’s comments: A critical look at the use of PLS-SEM in *MIS Quarterly*. *MIS Quarterly*, 36(1), iii–xiv.
- Ringle, C. M., Wende, S., & Will, A. (2005).** *SmartPLS 2.0*. Retrieved from <http://www.smartpls.de>
- Sabherwal, R., & Chan, Y. E. (2001).** Alignment between business and IS strategies: A study of prospectors, analyzers, and defenders. *Information Systems Research*, 12(1), 11–33.
- Sarker, S., Munson, C. L., Sarker, S., & Chakraborty, S. (2009).** Assessing the relative contribution of the facets of agility to distributed systems development success: An analytic hierarchy process approach. *European Journal of Information Systems*, 18(4), 285–299.
- Sarker, S., & Sarker, S. (2009).** Exploring agility in distributed information systems development teams: An interpretive study in an offshoring context. *Information Systems Research*, 20(3), 440–461.

- Schwaber, K., & Beedle, M. (2002).** *Agile software development with Scrum*. Upper Saddle River, NJ: Prentice Hall.
- Serrador, P., & Pinto, J. K. (2015).** Does agile work? A quantitative analysis of agile project success. *International Journal of Project Management*, 33(5), 1040–1051.
- Siau, K., Long, Y., & Ling, M. (2010).** Toward a unified model of information systems development success. *Journal of Database Management*, 21(1), 80–101.
- So, C., & Scholl, W. (2009).** Perceptive agile measurement: New instruments for quantitative studies in the pursuit of the social-psychological effect of agile practices. In P. Abrahamsson, M. Marchesi & F. Maurer (Eds.), *Agile processes in software engineering and extreme programming-XP2009* (Vol. 31, pp. 83–93). Pula, Italy: Springer.
- Straub, D. W., Boudreau, M.-C., & Gefen, D. (2004).** Validation guidelines for IS positivist research. *Communications of the Association for Information Systems*, 13(24), 380–427.
- Sutherland, J., & Altman, I. (2009, 24–28 August, 2009).** *Take no prisoners: How a venture capital group does Scrum*. Paper presented at the Agile Conference, 2009. AGILE '09, Chicago, Illinois.
- Sutherland, J., & Altman, I. (2010).** *Organizational transformation with Scrum: How a venture capital group gets twice as much done with half the work*. Paper presented at the 43rd Hawaii International Conference on System Sciences, Honolulu, Hawaii.
- Takeuch, H., & Nonaka, I. (1986).** The new new product development game. *Harvard Business Review*, 64(1), 137–146.
- Varela, F. J. (1984).** Two principles of self-organization. In H. Ulrich & G. J. B. Probst (Eds.), *Self-organization and management of social systems* (pp. 25–32). New York, NY: Springer.
- VersionOne. (2015).** The 9th Annual State of Agile™ Survey. Retrieved from <http://info.versionone.com/state-of-agile-development-survey-ninth.html>
- Vidgen, R., & Wang, X. (2009).** Coevolving systems and the organization of agile software development. *Information Systems Research*, 20(3), 355–376.
- Wallace, L., & Keil, M. (2004).** Software projects risks and their effect on outcomes. *Communications of the ACM*, 47(4), 68–73.
- Wallace, L., Keil, M., & Rai, A. (2004).** Understanding software project risk: A cluster analysis. *Information & Management*, 42(1), 115–125.
- Wang, X., Conboy, K., & Cawley, O. (2012).** "Leagile" software development: An experience report analysis of the application of lean approaches in agile software development. *Journal of Systems and Software*, 85(6), 1287–1299.
- Weitzel, J. R., & Graen, G. B. (1989).** System development project effectiveness: Problem-solving competence as a moderator variable. *Decision Sciences*, 20(3), 507–531.
- Wells, H. (2012).** How effective are project management methodologies? An explorative evaluation of their benefits in practice. *Project Management Journal*, 43(6), 43–58.
- Wetzels, M., Odekerken-Schröder, G., & Van Oppen, C. (2009).** Using PLS path modeling for assessing hierarchical construct models: Guidelines and empirical illustration. *MIS Quarterly*, 33(1), 177–195.
- Whitworth, E., & Biddle, R. (2007).** *Motivation and cohesion in agile teams*. Paper presented at the 8th International Conference on Agile Processes in Software Engineering and Extreme Programming, Como, Italy.
- Williams, L. (2012).** What agile teams think of agile principles. *Communications of the ACM*, 55(4), 71–76. doi: 10.1145/2133806.2133823
- Winter, S., Berente, N., Howison, J., & Butler, B. S. (2014).** Beyond the organizational 'container': Conceptualizing 21st century sociotechnical work. *Information and Organization*, 24(4), 250–269.
- Wright, R. T., Campbell, D. E., Thatcher, J. B., & Roberts, N. (2012).** Operationalizing multidimensional constructs in structural equation modeling: Recommendations for IS research. *Communications of the Association for Information Systems*, 30(23), 367–412.
- Xiaohu, Y., Xu, B., He, Z., & Maddineni, S. R. (2004).** *Extreme programming in global software development*. Paper presented at the Canadian Conference on Electrical and Computer Engineering, Niagara Falls, Canada.
-
- Jan Recker, PhD**, is Professor in the QUT Business School at Queensland University of Technology, Brisbane, Australia. His research focuses on process analysis and design, digital innovation, and environmental sustainability. He has published in the *MIS Quarterly*, *Journal of the Association for Information Systems*, *Information Systems Journal*, *Academy of Management Discoveries*, and elsewhere. He can be contacted at j.recker@qut.edu.au
-
- Roland Holten, PhD**, is Professor in the Faculty of Economics and Business Administration, Institute for Business Informatics at Goethe University, Frankfurt, Germany. His research focuses on communication structures of groups and information systems engineering. He has published in the *European Journal of Information Systems*, *Information Systems Journal*, *Journal of Information Technology*, *Journal of the Association for Information Systems*, and elsewhere. He can be contacted at holten@wiwi.uni-frankfurt.de
-
- Markus Hummel, PhD**, is senior business IT consultant with Senacor Technologies AG. He received a doctoral degree from Goethe University, Frankfurt, Germany. His research focuses on agile information systems development. He has published articles in such publications as *Communications of the Association for Information Systems* and *Business & Information Systems Engineering*. He can be contacted at Markus.Hummel@senacor.com
-
- Christoph Rosenkranz, PhD**, is Professor in the Faculty of Economics, Management and Social Sciences at the University of Cologne, Cologne, Germany. His research focuses on business process management, information systems development, and IT project management. He has published articles in publications, including *European Journal of Information Systems*, *Information Systems Journal*, *Journal of Information Technology*, and *Journal of the Association for Information Systems*. He can be contacted at rosenkranz@wiso.uni-koeln.de

How Agile Practices Impact Customer Responsiveness and Development Success

Appendix A: Construct measurement.

Construct	Code	Item	Source
Stand-up meeting	STM1	Stand-up meetings are extremely short (maximum 15 minutes).	So and Scholl (2009)
	STM2	Stand-up meetings are to the point, focusing only on what has been done and what needs to be done on that day.	
	STM3	All relevant technical issues and organizational impediments come up in the stand-up meetings.	
	STM4	When people report problems in the stand-up meetings, team members offer assistance instantly.	
Collective code ownership	CC01	Anyone on the team can change existing code at any time.	Maruping, Zhang, and Venkatesh (2009)
	CC02	If anyone wanted to change a piece of code, he or she needed the permission of the individual(s) who coded it.	
	CC03	In our team, we feel comfortable changing any part of the existing code at any time.	
	CC04	A unit of code can only be changed by the individual who developed it.	
	CC05	In our team, we all feel a sense of responsibility for the system code.	
Pair programming	PP1	We do our software development using pairs of developers.	Maruping, Venkatesh, and Agarwal (2009)
	PP2	How often is pair programming used in the team?	
	PP3	To what extent is programming carried out by pairs of developers on the team?	
Software team response extensiveness		To what extent did the software team actually incorporate requirement changes in each of the following categories? (For example, if the project actually incorporated four out of ten different changes in a specific category, your answer would be 40%.)	Lee and Xia (2010)
	EXT1	System scope	
	EXT2	System input data	
	EXT3	System output data	
	EXT4	Business rules/processes	
	EXT5	Data structure	
	EXT6	User interface	
Software team response efficiency		How much additional effort was required by the software team to incorporate the following changes? (Effort includes time, cost, personnel, and resources.)	Lee and Xia (2010)
	EFF1	System scope	
	EFF2	System input data	
	EFF3	System output data	
	EFF4	Business rules/processes	
	EFF5	Data structure	
	EFF6	User interface	
Customer satisfaction (information systems development success)		How do the customers feel about the software that the team has developed?	Bhattacharjee (2001)
	CSF1	Very dissatisfied . . . Very satisfied.	
	CSF2	Very displeased . . . Very pleased.	
	CSF3	Very frustrated . . . Very contented.	
Process performance (information systems development success)	PPF1	The project was or will be completed within budget.	Lee and Xia (2010); Wallace et al. (2004)
	PPF2	The project was or will be completed within schedule.	

(continued)

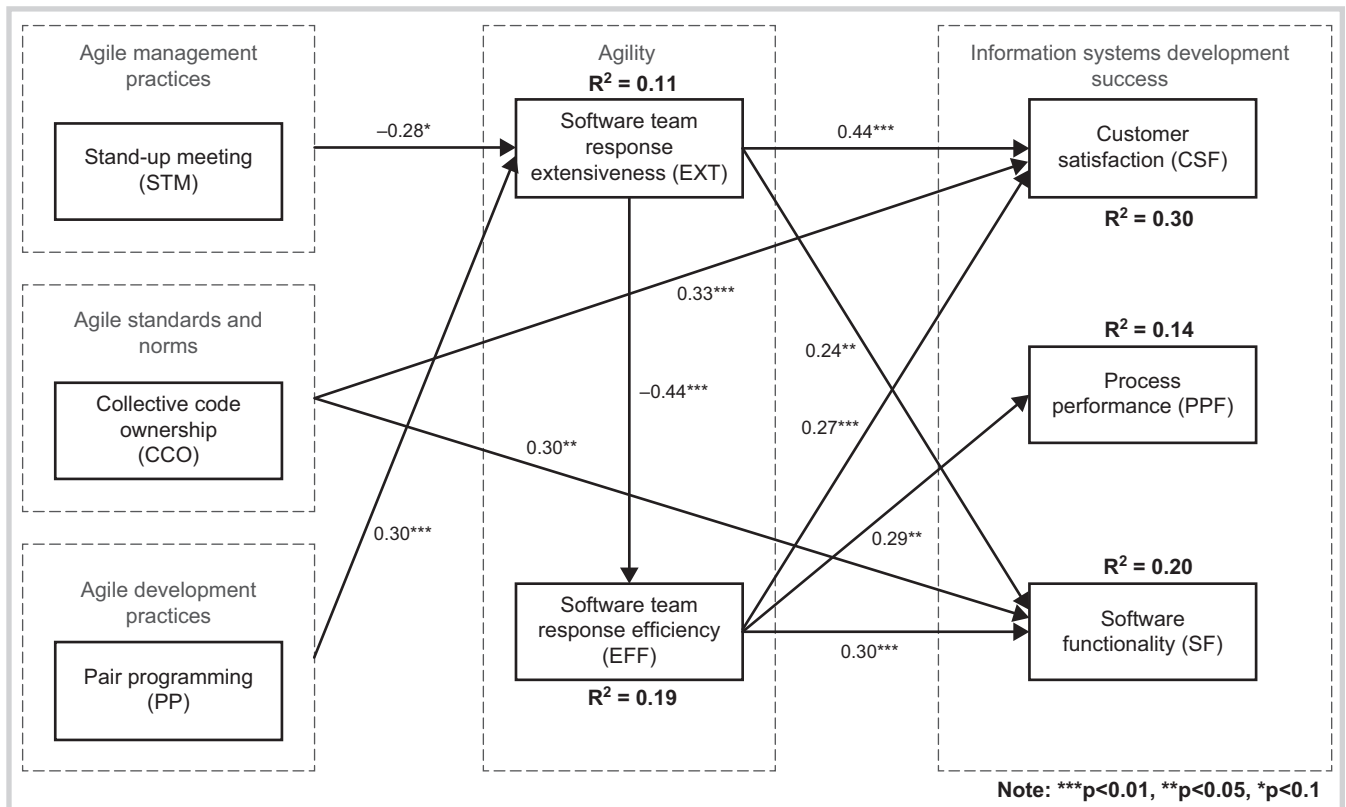
Construct	Code	Item	Source
Software dunctionality (information systems development success)	SF1	The software delivered by the project achieves or will achieve its functional goals.	Lee and Xia (2010)
	SF2	The software delivered by the project meets or will meet end-user requirements.	
	SF3	The capabilities of the software fit or will fit end-user needs.	
	SF4	The software meets or will meet technical requirements.	
Software team autonomy*	AUT01	The project team was allowed to freely choose tools and technologies.	Lee and Xia (2010)
	AUT02	The project team had control over what they were supposed to accomplish.	
	AUT03	The project team was granted autonomy on how to handle user requirement changes.	
	AUT04	The project team was free to assign personnel to the project.	
Software team diversity*	DIV1	The members of the project team were from different areas of expertise.	Lee and Xia (2010)
	DIV2	The members of the project team had skills that complemented each other.	
	DIV3	The members of the project team had a variety of different experiences.	
	DIV4	The members of the project team varied in functional backgrounds.	

* Measures used for control check purposes only.

How Agile Practices Impact Customer Responsiveness and Development Success

Appendix B: First-order structural model analysis.

We evaluated a first-order model with the three dimensions of information systems development success, customer satisfaction, process performance, and software functionality, as distinct constructs. The resulting structural model is presented in Appendix B, Figure 1. For the sake of clarity, we only include significant paths in Appendix B, Figure 1. Effect sizes are reported in Appendix B, Table 1. The results show that there is no direct significant effect of stand-up meetings and pair programming on any of the success dimensions. Collective code ownership is positively correlated with customer satisfaction and software functionality. Both the agility constructs, namely software team response extensiveness and efficiency, are significantly correlated with each of the success dimensions, but extensiveness does not significantly impact process performance. The effect sizes from collective code ownership to customer satisfaction and from software team responsive extensiveness to customer satisfaction and software team responsive efficiency are medium, and all other effects are small (see Appendix B, Table 1).



Appendix B, Figure 1: First-order model results.

Variable	EXT	EFF	CSF	PPF	SF
CCO			0.11		0.083
PP	0.092				
STM	0.061				
EXT		0.209	0.205	0.021	0.055
EFF			0.085	0.076	0.090

Appendix B, Table 1: Effect sizes for exogenous constructs in first-order model.

Appendix C: Model comparison.

Appendix C, Table 1 compares *our model* with *Lee and Xia's (2010) model* as well as with a third model that combines our model with the model of Lee and Xia (2010) (*integrated model*). For this analysis, we included the control measures software team autonomy and software team diversity (see Appendix A). Both measures proved to be reliable and valid.

The results shows that our model explains substantially more variance (R^2 change = 0.109) in information systems development success compared with Lee and Xia's (2010) model given the data collected. Also, our model explains more of the variance of software team response efficiency and a little less of the variance of the software team response extensiveness compared to Lee and Xia (2010). By adding the two constructs of software team autonomy and diversity to our model (the integrated model), the explained variance of information systems development success increases only marginally (from 0.305 to 0.308), and only the explained variance in software team response extensiveness is doubled. We interpret these results as suggesting our model is more parsimonious than the integrated model. In sum, the model comparison in Appendix C, Table 1 provides support for our theoretical argument that agility and information systems development success is more dependent on the used agile practices than on team characteristics such as autonomy or diversity.

Our Model			Lee and Xia's (2010) Model			Integrated Model		
Constructs and relations	Loadings	R^2	Constructs and relations	Loadings	R^2	Constructs and relations	Loadings	R^2
EXT		0.111	EXT		0.121	EXT		0.231
STM→EXT	−0.287**		AUTO→EXT	0.279 ^{ns}		STM→EXT	−0.333**	
CCO→EXT	0.135 ^{ns}		DIV→EXT	0.147 ^{ns}		CCO→EXT	0.338 ^{ns}	
PP→EXT	0.302***					PP→EXT	0.279**	
						AUTO→EXT	0.338 ^{ns}	
						DIV→EXT	0.058 ^{ns}	
EFF		0.190	EFF		0.157	EFF		0.194
STM→EFF	−0.189 ^{ns}		AUTO→EFF	−0.003 ^{ns}		STM→EFF	−0.200 ^{ns}	
CCO→EFF	0.209 ^{ns}		DIV→EFF	−0.059 ^{ns}		CCO→EFF	0.210 ^{ns}	
PP→EFF	0.095 ^{ns}		EXT→EFF	−0.378***		PP→EFF	0.115 ^{ns}	
EXT→EFF	−0.438***					EXT→EFF	−0.430***	
						AUTO→EFF	0.018 ^{ns}	
						DIV→EFF	0.082 ^{ns}	
ISDS		0.305	ISDS		0.196	ISDS		0.308
EXT→ISDS	0.376***		EXT→ISDS	0.413***		EXT→ISDS	0.380***	
EFF→ISDS	0.337***		EFF→ISDS	0.389***		EFF→ISDS	0.338***	
STM→ISDS	0.032 ^{ns}					STM→ISDS	0.032 ^{ns}	
CCO→ISDS	0.326**					CCO→ISDS	0.326**	
PP→ISDS	0.026 ^{ns}					PP→ISDS	0.026 ^{ns}	

Appendix C, Table 1: Model comparison.

This material has been reproduced with the permission of the copyright owner. Unauthorized reproduction of this material is strictly prohibited. For permission to reproduce this material, please contact PMI.